1

Web Services

Web service fundamentals

Includes content adapted from *API Design Pattern*, 2021, Manning Publications && *RESTful Java Web Services, 3rd Ed.* , 2017, Packt Publishing

1

HTTP – Hypertext Transfer Protocol

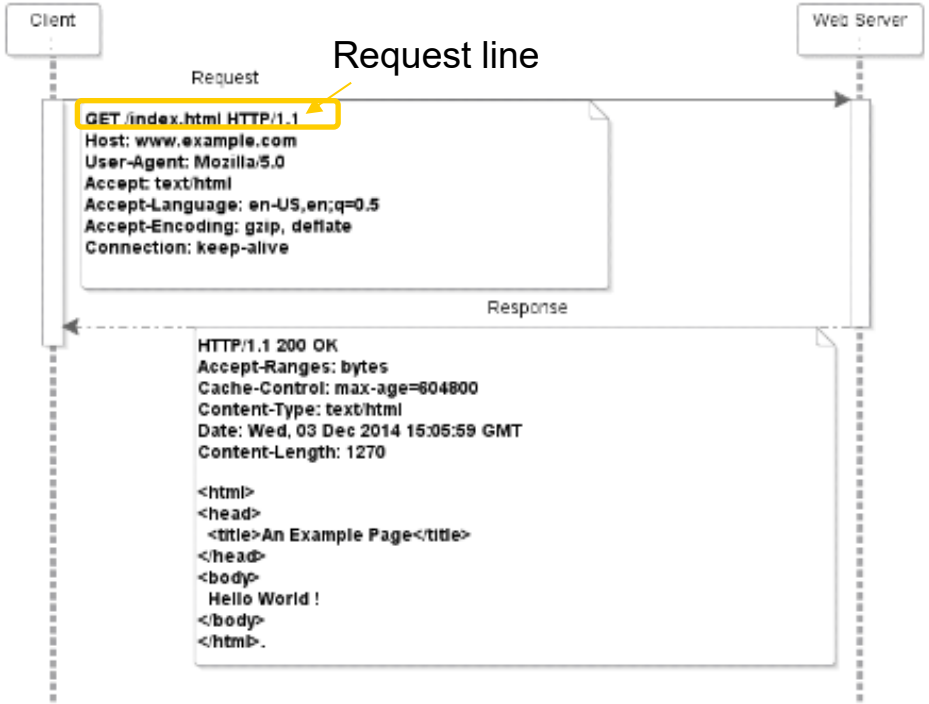
The foundation of data access and services on the Web:
A quick review/ crash course

Basic HTTP request-response

Command

GET /index.html HTTP/1.1

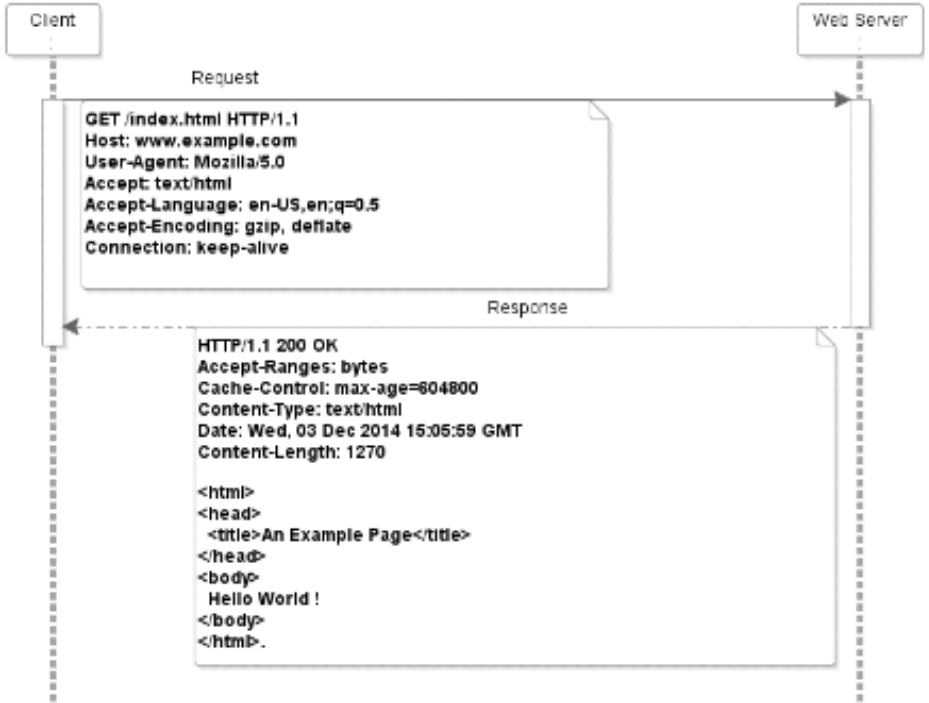
- The GET command requests some resource from the server
- It was the only command available in the initial version of the protocol
- Other commands were added in later versions



Basic HTTP request-response

Requested resource

```
GET /index.html HTTP/1.1
```



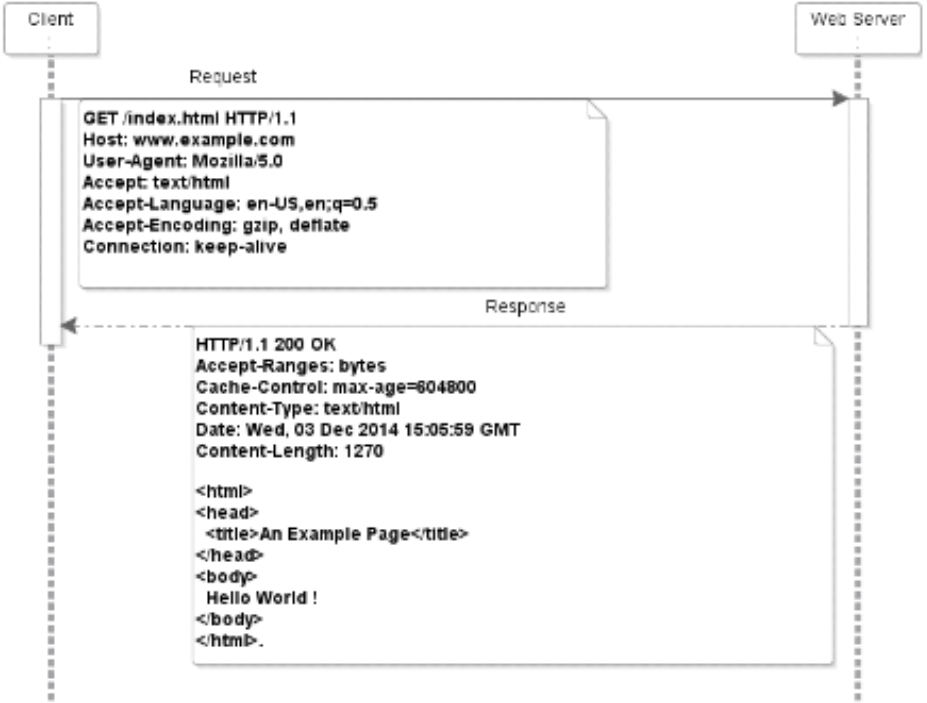
- The requested resource can refer to a static file available in the server.
- Alternatively, it may identify dynamically generated content.

Basic HTTP request-response

Client protocol version

GET /index.html HTTP/1.1

- The client identifies the version of the protocol it is implementing
- Necessary for allowing interoperability between clients and servers running potentially different protocol versions
- Original version is HTTP/0.9 (1991). HTTP/1.0 was introduced in 1996, then HTTP/2 in 2015, and currently HTTP/3 (2022) is the latest version.

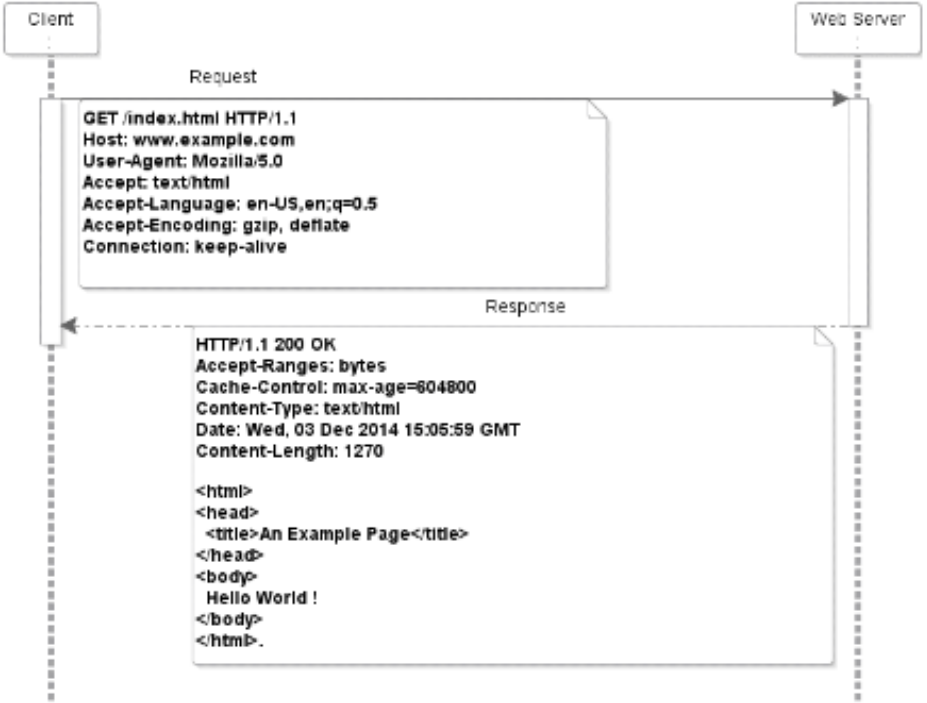


Basic HTTP request-response

CR+LF

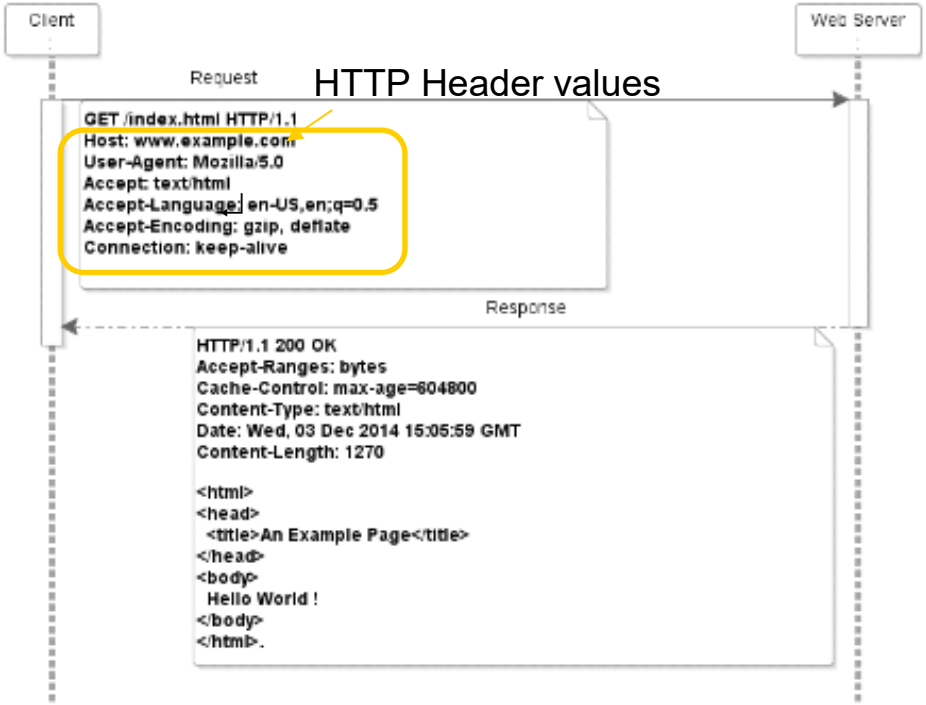
GET /index.html HTTP/1.1 ↵

- All the lines should be terminated with carriage return and line feed characters.



Basic HTTP request-response

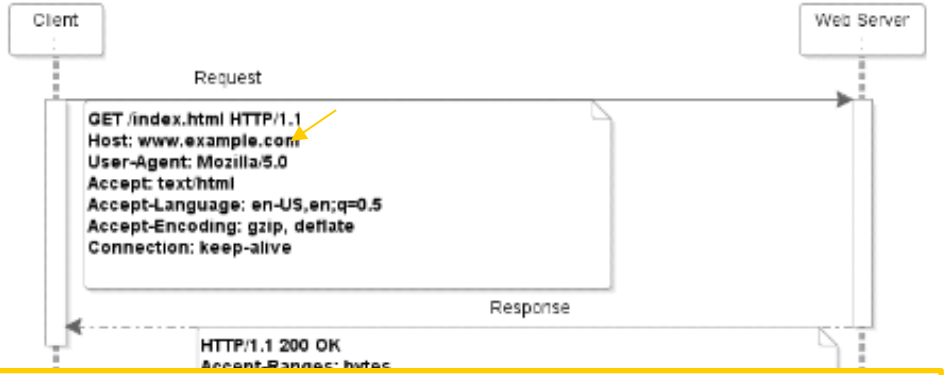
```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
```



- Header fields are colon-separated key-value pairs.
- They provide operating parameters to the server.
- A blank line is used at the end of the header to indicate the end of that section.

Basic HTTP request-response

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
```



- The client can specify the type of response it expects to receive using the Accept header
- The type of data is specified using MIME types (internet media type) using a type/subtype format

Common MIME types/subtypes

text: Plain text data (e.g. text/html)

image: image data (e.g. image/png)

audio: audio data (e.g audio/mpeg)

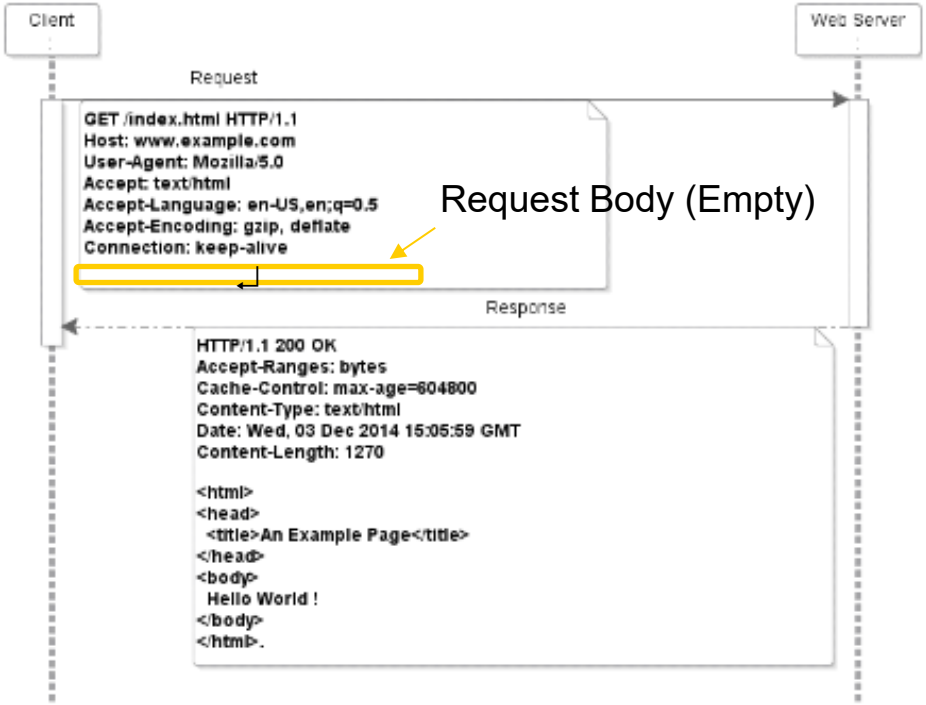
video: video data (e.g video/mp4)

multipart: multiple parts of data (e.g multipart/form)

application: Application data (e.g. application/json)

Basic HTTP request-response

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
```



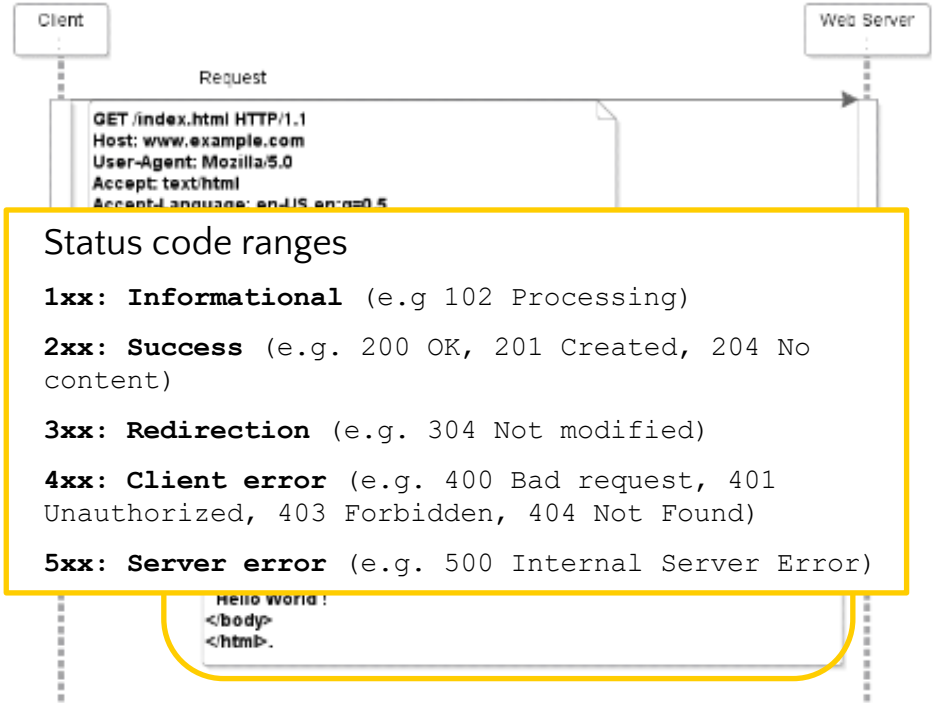
- Following the header and separated by a blank line, we find the request body. It is empty in this example.
- The request body is used when the client wants to send some data to the server

Basic HTTP request-response

Status Line

HTTP/1.1 200 OK

- The status line informs the client of the protocol being used by the server, and provides a status description, using a numeric code (“200”) followed by a textual representation of the status (“OK”)



Status code ranges

1xx: Informational (e.g 102 Processing)

2xx: Success (e.g. 200 OK, 201 Created, 204 No content)

3xx: Redirection (e.g. 304 Not modified)

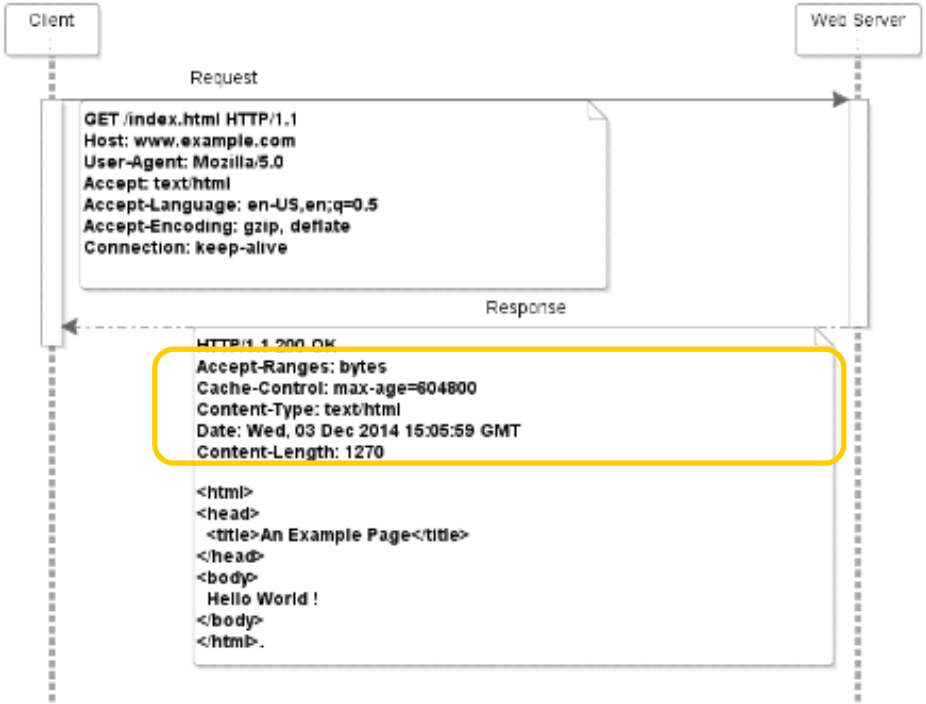
4xx: Client error (e.g. 400 Bad request, 401 Unauthorized, 403 Forbidden, 404 Not Found)

5xx: Server error (e.g. 500 Internal Server Error)

Basic HTTP request-response

Response headers

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Cache-Control: max-age=604800
Content-Type: text/html
Date: Wed, 03 Dec 2014 15:05:59 GMT
Content-Length: 1270
```



- The response headers are handled similarly to the request headers, and provide relevant meta information to the client.

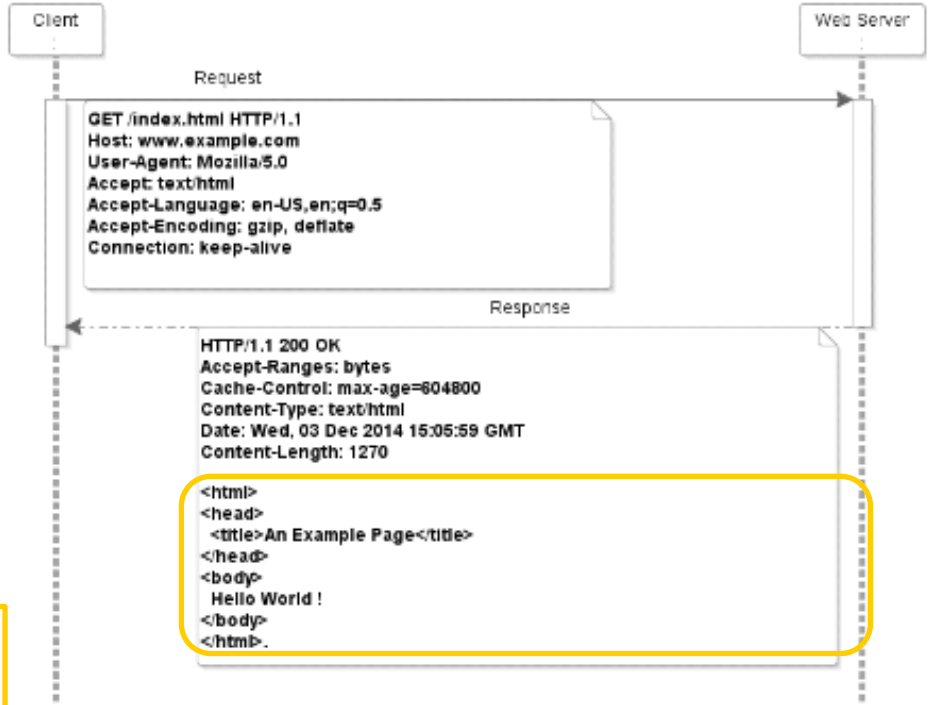
Basic HTTP request-response

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Cache-Control: max-age=604800
Content-Type: text/html
Date: Wed, 03 Dec 2014 15:05:59 GMT
Content-Length: 1270
```

Response Body

```
<html>
<head>
<title>An Example Page</title>
</head>
<body>
Hello World !
</body>
</html>
```

- The response body follows the response headers after a blank line.

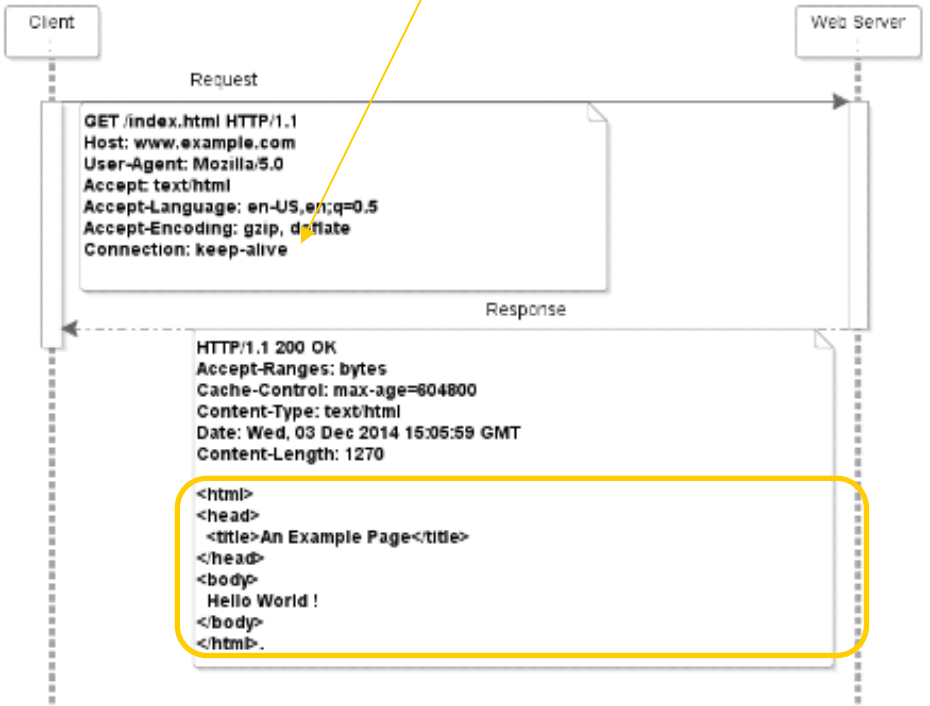


Basic HTTP request-response

After sending the response the server will disconnect unless the client requested to keep the connection alive

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Cache-Control: max-age=604800
Content-Type: text/html
Date: Wed, 03 Dec 2014 15:05:59 GMT
Content-Length: 1270

<html>
<head>
<title>An Example Page</title>
</head>
<body>
Hello World !
</body>
</html>
```





HTTP Request methods

Method	Description
GET	This method is used for retrieving resources from the server by using the given URI.
HEAD	This method is the same as the GET request, but it only transfers the status line and the header section without the response body.
POST	This method is used for posting data to the server. The server stores the data (entity) as a new subordinate of the resource identified by the URI. If you execute POST multiple times on a resource, it may yield different results.
PUT	This method is used for updating the resource pointed by the URI. If the URI does not point to an existing resource, the server can create the resource with that URI.
DELETE	This method deletes the resource pointed by the URI.
TRACE	This method is used for echoing the contents of the received request. This is useful for the debugging purpose with which the client can see what changes (if any) have been made by the intermediate servers.
OPTIONS	This method returns the HTTP methods that the server supports for the specified URI.
CONNECT	This method is used for establishing a connection to the target server over HTTP.
PATCH	This method is used for applying partial modifications to a resource identified by the URI.

2

Web API Basics

Fundamentals of using web services API over HTTP



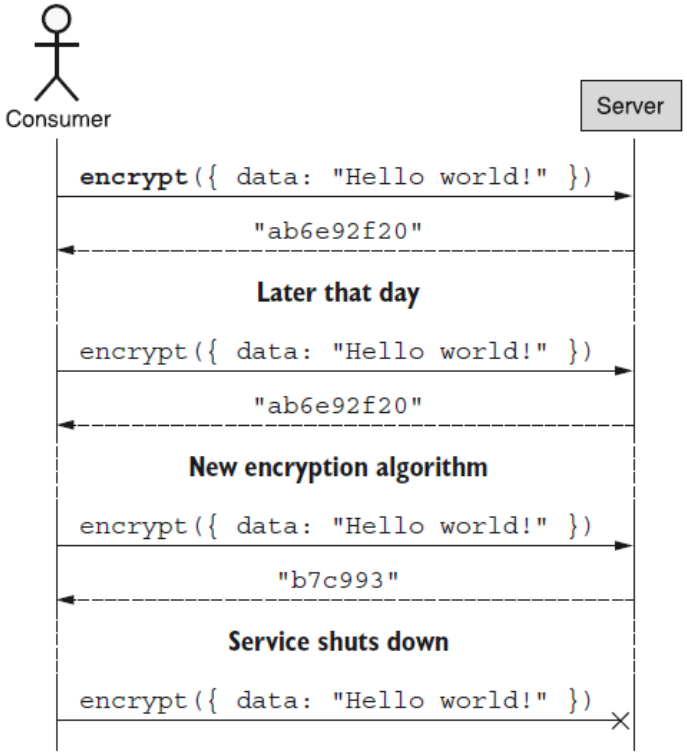
What are Web APIs?

- An API (Application Programming Interface) defines how systems interact.
  - A library of a programming language defines an API
  - You define your own API when creating your classes and methods..
- Web APIs, specifically, are APIs designed to be exposed over the network to be used remotely by clients.



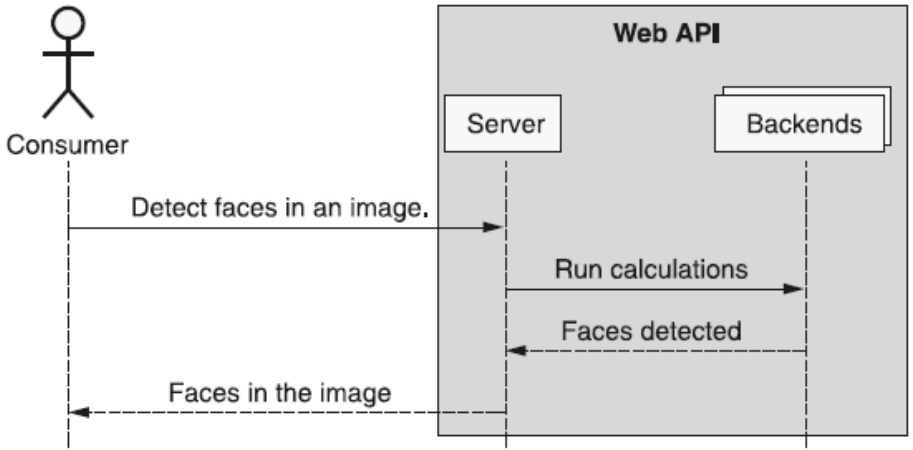
What are Web APIs?

- An Web API developer has complete control of the API evolution.
- Users of the API are forced to use whatever version of the API the developer has most recently published.
  - *No local copies…*



What are Web APIs?

- Web APIs expose functionality without distributing code
  - Great for implementing secret proprietary algorithms!
- They also allow users to tap into computational resources they would not have access otherwise.
- Web APIs allow automation and composition of systems.





Resource-Oriented APIs

- An API can be created by specifying arbitrary functions (procedures) much in the same way that these are defined when creating a computer program:
  - E.g. For obtaining a weather forecast, a user might call the `predictWeather(postalCode=3030)` service.
- This style of interaction is often referred to as RPC (Remote Procedure Call).
  - The primary focus is on the *actions* being performed.



Resource-Oriented APIs

- An Example of an RPC-style flight booking API

Method	Description
`ScheduleFlight()`	Schedules a new flight
`GetFlightDetails()`	Shows information about a specific flight
`ShowAllBookings()`	Shows all travel plans currently booked
`CancelReservation()`	Cancels an existing flight reservation
`RescheduleFlight()`	Reschedules an existing flight to another date or time
`UpgradeTrip()`	Upgrades from economy class to first class



Resource-Oriented APIs

- Each RPC is descriptive, but the user must memorize each one of these methods to be able to use the API.
  - Some methods operate on flights, other operate on reservations...
- There is no standard way of doing things,
  - e.g. for listing all the flights we might have `ShowFlights()`, `ShowAllFlights()`, `ListFlights()`, `ListAllFlights()`,...



Resource-Oriented APIs

- Resource-orientation addresses this issue by focusing on providing a standard set of two building blocks for API design:
  - **The Resources** – These are the “things” and key concepts the user interacts with (Flights, Reservations)
  - **The Standard Set of Actions** – A small set of common actions is made available, in the same way, for the relevant resources.
- RPC-Orientation has its place, but Resource-orientation makes the APIs easier to understand and use.



Resource-Oriented APIs

- Example of Resource-oriented API for flight booking API *(and RPC-style equivalent methods)*

Method		Resource		Methods
Create				CreateFlightReservation()
Get				GetFlightReservation()
List	×	FlightReservation	=	ListFlightReservations()
Delete				DeleteFlightReservation()
Update				UpdateFlightReservation()



What makes an API “good”?

- A good API should be:
  - **Operational:** The entire system does what it’s supposed to do
  - **Expressive:** The user can easily express what he wants to do…
  - **Simple:** Easy to do common things, but possible to do complex things.
  - **Predictable:** It is consistent in form and behavior.



What is a good API?

- **Operational**: The system as a whole must do the things its users want.
  - To do this, the system must satisfy both its functional AND non-functional requirements.
- ***Example:*** *A translation service must be capable of translating text from one language to another, and also do so in a timely and accurate way.*



What makes an API “good”?

- **Expressive**: The users must be able to express what they want to do simply.
  - It is often the case that the system has some built-in functionality that that is useful to the users that is not properly exposed in the API.
  - This leads to workarounds and inefficient use of the API.
- ***Example of inefficient workaround:*** *In a language translation system, a user interested in detecting the language a text is written in, for which no API support is provided, might repeatedly call the API, iterating over all supported languages, until the original and translated text are the same…*



What makes an API “good”?

- **Simple**: Expose the functionality the users want in the most straightforward way.
  - Not the same as reducing the number of RPCs or Resources.
  - Make the common case “awesome”, the advanced case “possible”.
    - E.g. *Use sensible defaults for rarely changed parameters.*
  - Do not obfuscate and hide things.
    - Provide the functionality the users may want directly and independently of other uses. Avoid relying on collateral effects for interacting with user (*e.g. the detected language is returned only as a part of the response to a text translation request).*



What makes an API “good”?

- **Predictable**: The API should not surprise the user, but behave exactly as he or she expects.
  - Users rarely “read the fabulous manual” (RTFM). They often read the bare minimum and learn by copying, adapting and trial and error.
  - Make sure that when users make “educated guesses” about the API they get it right.
  - Rely on consistent patterns for naming and behaviour.

  (e.g. If a field is named `text` in one message, then it would be a poor choice to have an `inputText` field in another message)